

SECTION A: BASIC PROGRAM STRUCTURE & I/O (Questions 1–6)

Problem 1: Digital Business Card Write a Java program that stores your full name (String), student ID (int), GPA (double), department initial (char), and whether you're currently enrolled (boolean). Print all values in a formatted business card style with borders using `System.out.println`.

Name	: Ahmed Ali
ID	: 20240001
GPA	: 3.75
Dept	: C
Active	: true

Problem 2: Data Type Limits Explorer Declare one variable of each primitive type: byte, short, int, long, float, double, char, boolean. Print the **maximum and minimum values** of each numeric type using wrapper class constants (e.g., Byte.MAX_VALUE, Integer.MIN_VALUE). Also print the size in bits of each type.

Problem 3: Currency Converter Accept an amount in Egyptian Pounds (EGP) as a command line argument. Convert and display the equivalent in:

- US Dollar (USD) — 1 USD = 48.5 EGP
- Euro (EUR) — 1 EUR = 52.3 EGP
- Saudi Riyal (SAR) — 1 SAR = 12.9 EGP

Use `Double.parseDouble()` and print each result formatted to 3 decimal places using `printf`.

Problem 4: Memory Usage Estimator A company stores employee records. Each record has: name (assume 50 chars = 50 bytes), age (int), salary (double), department code (short), and active status (boolean). Accept the number of employees as a command line argument. Calculate and display the **total memory in bytes, KB, and MB** needed to store all records.

Problem 5: Quadratic Equation Coefficients Accept three coefficients a , b , c of a quadratic equation ($ax^2 + bx + c = 0$) as command line arguments. Calculate and print the discriminant ($b^2 - 4ac$). Print whether the equation has:

- Two real roots (discriminant > 0)
- One real root (discriminant $= 0$)
- No real roots (discriminant < 0)

Do **not** calculate the roots yet — just the discriminant and the conclusion.

Problem 6: Receipt Generator Accept 4 item prices as command line arguments (doubles). Calculate:

- Subtotal (sum of all items)
- Discount: 10% if subtotal > 500, else 5%
- Tax: 14% on the discounted price
- Final total

Print a formatted receipt showing each line item with label and value aligned using printf with %-20s and %8.2f format specifiers.

SECTION B: OPERATORS & CALCULATIONS (Questions 7–13)

Problem 7: Power Calculator Without Math.pow Accept a base (int) and exponent (int) as command line arguments. Calculate $\text{base}^{\text{exponent}}$ using **only the multiplication operator** * inside a loop. Do not use Math.pow(). Display the result. Handle the case where exponent is 0 (result = 1). Use long to store the result.

Problem 8: Digit Extractor Accept a 5-digit integer from command line arguments. Using **only arithmetic operators** (no strings, no arrays), extract and print:

- Units digit
 - Tens digit
 - Hundreds digit
 - Thousands digit
 - Ten-thousands digit
 - Sum of all digits
 - Product of all digits
-

Problem 9: Bitwise Flag System Simulate a permission system using bit flags. Define 4 permissions using left-shift:

- READ = $1 \ll 0$ (bit 0)
- WRITE = $1 \ll 1$ (bit 1)
- EXECUTE = $1 \ll 2$ (bit 2)
- DELETE = $1 \ll 3$ (bit 3)

Accept a permission integer (0–15) as command line argument. Use bitwise AND & to check which permissions are set and print them. Use bitwise OR | to grant all permissions and print the result.

Problem 10: Number Sign Analyzer Accept any integer. Without using if statements or ternary operators — using **only arithmetic and bitwise operators** — determine if the number is positive, negative, or zero. Hint: use right shift `>>` by 31 for sign extraction. Print the result with an explanation of each step.

Problem 11: Compound Interest Comparator Accept principal (double), annual rate (double), and years (int) from command line. Calculate and compare:

- **Simple Interest:** $SI = P \times R \times T / 100$
 - **Compound Interest (annually):** $CI = P \times (1 + R/100)^T$ — calculate $(1+R/100)^T$ using repeated multiplication in a loop, not `Math.pow()`
 - Print the difference and which one yields more profit.
-

Problem 12: Digital Root Calculator The digital root of a number is found by repeatedly summing its digits until you get a single digit (e.g., $9875 \rightarrow 9+8+7+5 = 29 \rightarrow 2+9 = 11 \rightarrow 1+1 = 2$). Accept a number from command line. Use the modulus `%` and division `/` operators. Print each step of the process and the final digital root.

Problem 13: Operator Precedence Demonstrator Without using any parentheses, write expressions that demonstrate operator precedence. Pre-define `int a=6, b=4, c=2, d=3`. Evaluate and print each of the following, then manually verify:

- `a + b * c - d`
- `a / b + c * d`
- `a % b + c << 1`
- `a > b && b > c || d == 3`
- `a & b | c ^ d`

Print the expression as a String alongside the result.

SECTION C: DECISION MAKING (Questions 14–21)

Problem 14: Smart BMI Calculator Accept weight (kg) and height (meters) as command line arguments. Calculate $BMI = \text{weight} / (\text{height} \times \text{height})$. Use an if-else-if ladder to classify:

- Below 18.5: Underweight
- 18.5–24.9: Normal weight
- 25.0–29.9: Overweight
- 30.0 and above: Obese

Also use a ternary operator to print whether the person needs a "Diet Plan" or "Maintain Current Diet".

Problem 15: Traffic Light Simulator Accept a color ("RED", "YELLOW", "GREEN") as a command line argument. Use a switch statement with **arrow syntax (Java 14+)** to print:

- RED → "STOP — Wait for green"
- YELLOW → "SLOW DOWN — Prepare to stop"
- GREEN → "GO — Proceed safely"
- Default → "INVALID SIGNAL"

Convert input to uppercase first using `.toUpperCase()` before the switch.

Problem 16: Shipping Cost Calculator A courier company charges based on weight (kg) and destination zone (1–4). Accept both as command line arguments.

Zone Up to 5kg 5–20kg Above 20kg

1	25	40	60
2	35	55	80
3	50	75	110
4	70	100	150

Use nested if-else (outer = zone, inner = weight range). Print the shipping cost and a confirmation message.

Problem 17: Palindrome Number Checker Accept an integer from command line. Using only arithmetic operators (`%`, `/`), reverse the number. Use an if-else statement to compare the reversed number with the original and print whether it is a palindrome. Handle negative numbers (they are never palindromes).

Problem 18: Season and Weather Advisor Accept month number (1–12) and temperature (int, in Celsius) from command line. Use a switch-case for the month to determine the season (Spring/Summer/Autumn/Winter). Then use nested if-else on temperature to print a clothing advice message. Example: Season=Summer, Temp=38 → "Wear light cotton clothes and stay hydrated."

Problem 19: ATM Simulator (Decision Only) Accept account balance (double), withdrawal amount (double), and PIN entered (int) as command line arguments. Correct PIN is 1234. Use nested if-else to:

- Check PIN first (print "Wrong PIN" and stop if incorrect)
 - Check if withdrawal amount is a multiple of 100 (ATMs dispense in 100s)
 - Check if balance is sufficient (minimum balance = 500 EGP must remain)
 - If all pass, print new balance; otherwise print specific error message.
-

Problem 20: Roman Numeral Converter (1–39) Accept a number between 1 and 39 from command line. Use a switch-case for tens (10, 20, 30) and another for units (1–9) to build the Roman numeral string by concatenation. Print the Roman numeral. Use a default case for numbers outside 1–39.

Problem 21: Electricity Bill Calculator Accept units consumed (int) from command line. Calculate bill using slabs:

- 0–100 units: 0.50 EGP/unit
- 101–200 units: 0.75 EGP/unit (first 100 at 0.50)
- 201–500 units: 1.20 EGP/unit (previous slabs apply)
- Above 500: 1.50 EGP/unit

Use if-else-if. Add a fixed service charge of 50 EGP. Apply a 15% surcharge if total bill exceeds 500 EGP. Print a detailed bill breakdown.

SECTION D: LOOPS (Questions 22–30)

Problem 22: Prime Sieve (Brute Force) Print all prime numbers between 1 and 500 using nested loops. For each number, check divisibility from 2 to its square root using `Math.sqrt()`. Print primes in rows of 10 separated by tabs. At the end, print the **count** of primes found and their **sum**.

Problem 23: Number Pattern — Diamond Accept an odd number `n` from command line (e.g., `n=7`). Print a diamond pattern using nested for loops with spaces and stars. Example for `n=5`:

```
*  
  
***  
  
*****  
  
***  
  
*
```

The top half uses an increasing loop, the bottom half a decreasing loop.

Problem 24: Collatz Conjecture Accept any positive integer from command line. Apply the Collatz rules using a while loop:

- If even: divide by 2
- If odd: multiply by 3 and add 1
- Stop when you reach 1

Print every step of the sequence. Count and print the number of steps taken (called the "stopping time").

Problem 25: Number Base Converter Accept a decimal integer from command line. Convert it to:

- **Binary** (base 2): use % 2 and / 2 in a while loop, build string by prepending digits
- **Octal** (base 8): use % 8 and / 8
- **Hexadecimal** (base 16): digits 0–9 then A–F

Do **not** use Integer.toBinaryString() or similar methods. Build each conversion manually. Print all three results.

Problem 26: Pascal's Triangle Accept number of rows n from command line. Print Pascal's Triangle using nested loops. Calculate each value using the formula: $\text{value} = \text{previous_value} * (\text{col}) / \text{row}$ to avoid needing arrays for computation. Format with appropriate leading spaces.

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

Problem 27: GCD and LCM Calculator Accept two positive integers from command line. Use a while loop to implement the **Euclidean algorithm** to find GCD: repeatedly replace the larger number with larger % smaller until one becomes 0. Calculate LCM using the formula: $\text{LCM} = (a \times b) / \text{GCD}$. Print each step of the GCD calculation, then the final GCD and LCM.

Problem 28: Number Guessing Game Simulator Pre-define a secret number (e.g., int secret = 37). Pre-define an array of guesses to simulate user input. Use a do-while loop to process each guess:

- Print "Too High" / "Too Low" / "Correct!"
- Count number of attempts
- Stop when correct or all guesses exhausted

Print a summary: number of attempts and whether the game was won or lost.

Problem 29: Multiplication Grid Print a 10×10 multiplication table grid using nested for loops. Format each cell as a 4-character wide number using printf("%4d", ...). Print column headers (1–10) and row headers. Add a separator line after the header row using a loop printing dashes.

Problem 30: Sum of Series Calculate and print the sum of the following series up to n terms (accept n from command line). Use a for loop for each:

- **Series A:** $1 + 1/2 + 1/3 + 1/4 + \dots + 1/n$ (Harmonic)
- **Series B:** $1^2 + 2^2 + 3^2 + \dots + n^2$ (Sum of squares)

- **Series C:** $1 - 1/3 + 1/5 - 1/7 + \dots$ (Leibniz formula — multiply result by 4 to approximate π)

Print results to 6 decimal places. For Series C, also print the approximation of π obtained.

SECTION E: ARRAYS & STRINGS (Questions 31–40)

Problem 31: Array Rotation Create an int array of 8 elements initialized with values 10–80. Write logic using a loop to **rotate the array left by k positions** (accept k from command line). Print the array before and after rotation. Do not use a second full-size array — use a temporary variable approach.

Problem 32: Frequency Counter Accept a String sentence from command line. Convert it to lowercase. Create an int array of size 26 (for a–z). Loop through each character using `charAt()` and increment the corresponding array index using `ch - 'a'`. Print a frequency table showing each letter and how many times it appears (skip letters with count 0).

Problem 33: String Compression Accept a String from command line. Implement **Run-Length Encoding**: consecutive repeated characters are replaced by the character followed by its count (e.g., "aaabbbcc" → "a3b3c2", "abc" → "a1b1c1"). Use a while loop with an index. Build result using String concatenation. Print original and compressed strings. Also print which is shorter.

Problem 34: Matrix Multiplication Declare two 3×3 int matrices hardcoded with values. Perform matrix multiplication using triple nested for loops (the standard $O(n^3)$ algorithm). Print both input matrices and the resulting product matrix. Format each matrix nicely with aligned columns using `printf`.

Problem 35: Anagram Checker Accept two Strings from command line. Use int arrays of size 26 to count character frequencies in each string (ignore spaces, convert to lowercase). Compare the two frequency arrays element by element. Print whether the strings are anagrams of each other. Example: "listen" and "silent" are anagrams.

Problem 36: Student Grade Report Create a parallel array setup: one String array for 6 student names, one 2D int array for their marks in 5 subjects. Hardcode the data. Using loops, calculate for each student: total, average, and grade (A/B/C/D/F). Print a formatted table. Then find and print the overall class topper (highest average).

Problem 37: Wrapper Class Conversion Chain Demonstrate the following conversion chain without using casting syntax (int):

- String → int using `Integer.parseInt()`
- int → double using Integer autoboxing then assignment
- double → String using `Double.toString()`
- String → boolean using `Boolean.parseBoolean()`

- int → binary String using Integer.toBinaryString()
- String → char array using .toCharArray()

For each step, print the value and its type name to demonstrate the conversion.

Problem 38: Sparse Matrix Checker Create a 4×5 int matrix (hardcoded with many zeros). A matrix is "sparse" if more than half its elements are zero. Count zero elements using nested loops. Print the matrix, the count of zeros vs non-zeros, and whether it is sparse. Also print the **density** (ratio of non-zero elements to total) as a percentage.

Problem 39: Word Reverser Accept a sentence (multiple words) as command line arguments — each word will be a separate element in String[] args. Using a for loop that goes **backwards** through the args array, reconstruct the sentence in reverse word order. Also, for each word, reverse its characters using a char-level loop. Print:

- Original sentence
 - Sentence with reversed word order
 - Sentence with each word's characters reversed
-

Problem 40: Number to Binary String Art Accept a number 1–15 from command line. Convert it to a 4-bit binary representation (using %2 and /2 in a loop, storing in a char array of size 4). Then print a "pixel art" where 1 =  and 0 = (two spaces). Print 5 such rows to create a solid or hollow visual block. Label each row with its bit value (8, 4, 2, 1).

SECTION F: CONTROL FLOW & JUMP STATEMENTS (Questions 41–46)

Problem 41: Prime Pair Finder (Labeled Loops) Print all **twin prime pairs** between 1 and 200 (twin primes differ by 2, e.g., (3,5), (11,13)). Use an outer labeled for loop over all numbers and an inner loop for primality checking. Use a continue with label to skip to the next candidate when a divisor is found. Count and print the total number of twin prime pairs found.

Problem 42: Treasure Hunt Grid Create a 6×6 int grid. Place "treasure" at position (3,4) with value 999, and "traps" (value -1) at 5 random positions (hardcode them). Search the grid using nested loops with a **labeled break** when treasure is found. Use continue (with label) to skip traps and print "TRAP!" when encountered. Print the path taken and the final result.

Problem 43: Input Sanitizer with Continue Initialize a String array with 15 values: a mix of valid integers (as strings), negative numbers, empty strings "", strings with letters, and nulls. Loop through the array. Use continue to skip:

- null values
- empty strings

- strings containing non-digit characters

For valid positive integer strings, parse and sum them. Print skipped count, valid count, and the final sum.

Problem 44: Method Chain — Number Classifier Write **four separate static methods** (each using a return statement):

- boolean isEven(int n) — returns true if even
- boolean isDivisibleBy3(int n) — returns true if divisible by 3
- boolean isPerfectSquare(int n) — returns true if square root is whole number
- String classify(int n) — calls all three and returns a combined description like "Even, Divisible by 3, Not a Perfect Square"

In main, loop from 1 to 30, call classify() for each number, and print a formatted table.

Problem 45: Early Exit Optimization Write a method int findFirst(int[] arr, String condition) that uses early return to find:

- First prime number
- First perfect square
- First number divisible by both 4 and 6

Test with an array of 20 hardcoded integers. Demonstrate that the method exits as soon as the first match is found by printing a "Checking index X" message inside the loop (you'll see it stops early). Call the method three times with different condition strings and print results.

Problem 46: Nested Loop Optimizer with Break Write a program to find all **Pythagorean triplets** ($a^2 + b^2 = c^2$) where a, b, c are all less than or equal to 100. Use three nested for loops (a, b, c). When a valid triplet is found, print it. Use continue to skip if a >= b (to avoid duplicates). Also track how many triplets were found. After finding 10 triplets, use a **labeled break** to exit all three loops.

SECTION G: MIXED CHALLENGE PROBLEMS (Questions 47–50)

Problem 47: Caesar Cipher Accept a message (String) and a shift value (int, 1–25) as command line arguments. Implement **Caesar cipher encryption** and decryption:

- Shift each letter forward by shift positions in the alphabet (wrapping Z→A using % 26)
- Preserve case (uppercase stays uppercase)
- Leave non-letter characters unchanged

Use charAt() in a for loop, char arithmetic, and String concatenation. Print original, encrypted, and decrypted (shift back) strings. Verify decrypted == original.

Problem 48: Statistics Calculator Create an int array of 15 values (hardcode them with variety). Calculate and print:

- Mean (average)
- Variance: $\sum (x_i - \text{mean})^2 / n$
- Standard Deviation: $\sqrt{\text{variance}}$ using `Math.sqrt()`
- Range: `max - min`
- Median: sort the array using a nested loop (bubble sort), then find middle element(s)
- Mode: find the value that appears most frequently using a frequency counting loop

Print each metric with its name and value formatted to 4 decimal places.

Problem 49: Text-Based Progress Bar Accept a task name (String) and total steps (int) from command line. Simulate a progress bar using a for loop with `Thread.sleep(100)` for animation. At each step, print on the same line (use `\r` carriage return):

Downloading: [██████████░░░░░░░░░░] 40% (4/10 steps)

Build the bar using a loop: print  for completed steps and  for remaining. After completion, print a "Done!" message on a new line. Use `System.out.print` (not `println`) inside the loop.

Problem 50: Mini Expression Evaluator Accept a mathematical expression as three command line arguments: operand1 operator operand2 (e.g., 15 % 4). Support operators: +, -, *, /, %, ^ (power). Use a switch on the operator string. For ^, calculate power using a loop (not Math.pow()). Validate both operands are numbers using a method boolean isNumber(String s) that loops through each character with Character.isDigit(). Handle division by zero. Print the expression and result formatted as: 15 % 4 = 3.